

(T4) DISEÑO FÍSICO DE BD. DDL

MÓDULO DE BASES DE DATOS



DISEÑO FÍSICO DE BD. DATA DEFINITION LANGUAGE (DDL)



ÍNDICE:

1. INTRODUCCIÓN
2. LENGUAJE SQL
3. DDL
4. CARACTERÍSTICAS DEL DDL
5. EJEMPLOS DE DDL EN POSTGRESQL
 - 5.1. TIPOS DE DATOS EN POSTGRESQL
 - 5.2. CREACIÓN DE TABLAS Y RESTRICCIONES
 - 5.3. MODIFICACIÓN DE TABLAS Y RESTRICCIONES
 - 5.4. COMANDO PARA VER ESTRUCTURA DE UNA TABLA
 - 5.5. BORRADO DE TABLAS
 - 5.6. CREACIÓN Y BORRADO DE ÍNDICES
 - 5.7. CREACIÓN Y BORRADO DE VISTAS
6. DICCIONARIO DE DATOS
7. BIBLIOGRAFÍA

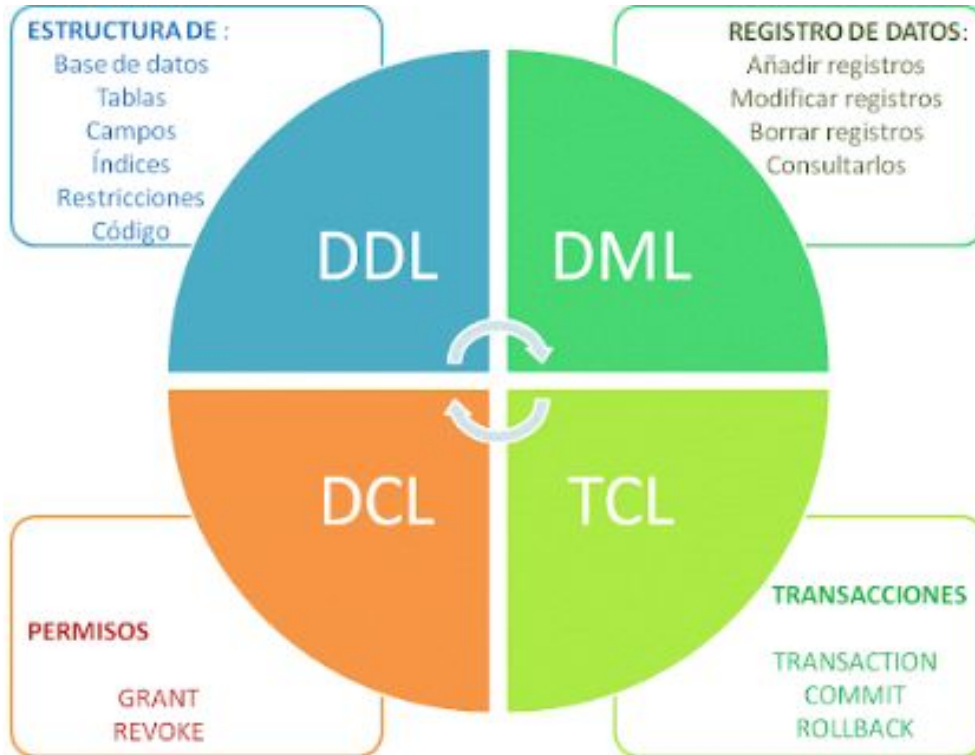
1. INTRODUCCIÓN

- A través del **diseño físico de la base de datos** se hace la **definición de datos** en los SGBD es el proceso mediante el cual **se especifican las estructuras que conforman la base de datos**. Este proceso es fundamental, ya que determina cómo se organizará la información y cómo se podrá acceder a ella de manera eficiente. El lenguaje más utilizado para definir estos aspectos en bases de datos relacionales es el Lenguaje de Definición de Datos o **DDL (Data Definition Language**, por sus siglas en inglés), que **se encarga de crear y modificar los objetos de la base de datos, como tablas, índices y vistas**.
- El DDL permite a los administradores de bases de datos crear esquemas que definen las características de los datos, tales como el tipo de datos, las relaciones entre las tablas y las restricciones de integridad. Por ejemplo, a través de comandos como **CREATE TABLE**, es posible definir una nueva tabla especificando sus atributos y tipos de datos.

2. LENGUAJE SQL

El **SQL** (Structured Query Language) es el estándar más ampliamente utilizado en bases de datos relacionales. Este lenguaje incluye tanto las capacidades del DDL como las del DML, así como DCL y TCL, lo que lo convierte en una herramienta integral para la creación, modificación y consulta de bases de datos, y también el control de la misma. Una de las ventajas más importantes del SQL es que **es un lenguaje declarativo**, es decir, los usuarios solo necesitan especificar qué datos desean obtener, y el SGBD se encarga de determinar cómo hacerlo.

2. LENGUAJE SQL



3. DDL

El **DDL** es la parte del lenguaje SQL que realiza la **función de definición de datos del SGBD**. Fundamentalmente, se encarga de la **creación, modificación y eliminación de los objetos de la base de datos** (es decir de los **metadatos**). Por supuesto es el encargado de la creación de las tablas.

Los elementos, llamados objetos, de la base de datos: **tablas, vistas, columnas, índices,... se almacenan en el diccionario de datos**. Por otro lado, muchos Sistemas Gestores de Bases de Datos aportan elementos para organizar estos objetos (como catálogos y esquemas).

Los objetos son manipulados y creados por los usuarios. En principio solo los administradores y los usuarios propietarios pueden acceder a cada objeto, salvo que se modifiquen los privilegios del objeto para permitir el acceso a otros usuarios.

Hay que tener en cuenta que **ninguna instrucción DDL puede ser anulada por una instrucción ROLLBACK** (la instrucción ROLLBACK está relacionada con el uso de transacciones, que se comentarán más adelante) por lo que hay que tener mucha precaución a la hora de utilizarlas. Es decir, **las instrucciones DDL generan acciones que no se pueden deshacer. Salvo que dispongamos de alguna copia de seguridad o de otros elementos de recuperación.**

3. DDL

El DDL se puede clasificar en dos grandes tipos en función de cómo se utiliza dentro del sistema:

- **DDL Interactivo:** Este tipo de DDL se utiliza de manera directa e interactiva, a menudo desde una consola de administración de bases de datos o un terminal SQL. Permite que los administradores de bases de datos ejecuten comandos y vean resultados de inmediato. Es útil para definir o modificar estructuras de datos de manera ad-hoc.
- **DDL Embebido:** Este tipo de DDL está incrustado dentro de programas escritos en lenguajes de programación de alto nivel (como C, Java o COBOL). Los comandos DDL se integran en el código fuente de la aplicación, y su ejecución es gestionada por el compilador o intérprete del lenguaje anfitrión. Esto permite la integración de la lógica de negocio con la manipulación de estructuras de datos.

4. CARACTERÍSTICAS DEL DDL

El DDL presenta una serie de características importantes que definen su uso en los sistemas de gestión de bases de datos:

1. **Simplicidad:** El DDL proporciona una manera directa y sencilla de definir y modificar estructuras de bases de datos. No requiere que el usuario especifique cómo se deben ejecutar las operaciones, lo que facilita su uso.
2. **Lenguaje de alto nivel:** El DDL es un lenguaje de alto nivel que se centra en describir qué estructuras de datos se necesitan, sin necesidad de entrar en los detalles de implementación.
3. **Dependencia con el SGBD:** El DDL está estrechamente ligado al Sistema de Gestión de Bases de Datos (SGBD) específico que se esté utilizando. Las características y capacidades del DDL pueden variar según el SGBD, aunque muchos de los comandos son estándares (como en SQL).
4. **Niveles de abstracción:** El DDL opera en tres niveles de abstracción: externo, lógico y físico. A nivel externo, permite definir las vistas de usuario; a nivel lógico, organiza la estructura general de la base de datos; y a nivel interno, controla cómo se almacenan físicamente los datos.

5. POSTGRESQL



PostgreSQL es un sistema de gestión de bases de datos relacional de código abierto y altamente avanzado. Reconocido por su robustez y cumplimiento de estándares SQL, PostgreSQL permite almacenar y gestionar grandes volúmenes de datos de manera segura y eficiente. Además de las funciones relacionales, admite características avanzadas como tipos de datos personalizados, JSON, consultas de texto completo, y operaciones de concurrencia, lo que lo convierte en una opción ideal para aplicaciones empresariales, web, y de análisis de datos.

Aplicación para utilizar PostgreSQL:

- **pgAdmin:** Es la aplicación gráfica oficial para la gestión de PostgreSQL. Ofrece una interfaz intuitiva para crear, modificar y administrar bases de datos, ejecutar consultas SQL, y gestionar usuarios y permisos.
- También puedes interactuar con PostgreSQL desde la **línea de comandos** utilizando **psql**, que es una herramienta incluida en las instalaciones de PostgreSQL para ejecutar consultas SQL y administrar la base de datos desde el terminal.

Ambas opciones, **pgAdmin** para una interfaz gráfica y **psql** para el terminal, permiten acceder y gestionar PostgreSQL de acuerdo con las necesidades del usuario.

5.0. CONEXIÓN DESDE CONSOLA (PSQL)



Usa el comando **psql** para conectarte

```
psql -h <host> -p <puerto> -U <usuario> -d <nombre_base_datos>
```

- Parámetros:
 - -h: Host o dirección del servidor (por defecto es `localhost`).
 - -p: Puerto del servidor PostgreSQL (por defecto es `5432`).
 - -U: Nombre de usuario para la conexión.
 - -d: Nombre de la base de datos a la que te deseas conectar.
- Si los valores son los predeterminados, puedes omitir algunos de ellos.

EJEMPLO BÁSICO DE CONEXIÓN:

```
psql -U usuario_ejemplo -d bd_ejemplo
```

5.0. CONEXIÓN DESDE CONSOLA (PSQL)



EJEMPLO BÁSICO DE CONEXIÓN:

```
psql -U usuario_ejemplo -d bd_ejemplo
```

```
SQL Shell (psql)
Server [localhost]:
Database [postgres]: bd_m
Port [5432]:
Username [postgres]: usuariom
Contraseña para usuario usuariom:
psql (17.2)
ADVERTENCIA: El código de página de la consola (850) difiere del código
de página de Windows (1252).
Los caracteres de 8 bits pueden funcionar incorrectamente.
Vea la página de referencia de psql «Notes for Windows users»
para obtener más detalles.
Digite «help» para obtener ayuda.

bd_m=> _
```

5.1. PRINCIPALES TIPOS DE DATOS EN POSTGRESQL

Tipo de Dato	Descripción	Ejemplo de Uso
INTEGER	Entero de 4 bytes, común para datos numéricos	edad INTEGER
SERIAL	Entero autoincremental, útil para claves primarias	id SERIAL PRIMARY KEY
VARCHAR(n)	Cadena de texto de longitud variable, hasta <code>n</code>	nombre VARCHAR(50)
TEXT	Cadena de texto de longitud ilimitada	descripcion TEXT
BOOLEAN	Valor lógico, admite <code>TRUE</code> , <code>FALSE</code> , <code>NULL</code>	es_activo BOOLEAN
DATE	Fecha sin información de hora	fecha_nacimiento DATE
TIMESTAMP	Fecha y hora sin zona horaria	fecha_registro TIMESTAMP
NUMERIC(p, s)	Precisión arbitraria para decimales (<code>p</code> =precisión, <code>s</code> =escala)	salario NUMERIC(10, 2)
JSON	Almacena datos en formato JSON, ideal para datos estructurados	datos JSON
UUID	Identificador único universal, útil para claves únicas	identificador UUID DEFAULT gen_random_uuid()



Nota: Hay muchos más tipos de datos, estos sólo son los más usuales.

5.2. EJEMPLOS DDL EN POSTGRESQL



Creación de Tablas y Restricciones Básicas

En PostgreSQL, el comando **CREATE TABLE** es el núcleo para definir tablas y columnas, especificando los tipos de datos y aplicando restricciones.

a) Clave Primaria (**PRIMARY KEY**)

La **clave primaria** es un identificador único de cada registro en una tabla. No permite valores nulos y asegura que cada valor en la columna es único.

id SERIAL PRIMARY KEY

- **SERIAL**: Tipo de dato especial que genera automáticamente un número único incremental, útil para definir una clave primaria.
- La restricción **PRIMARY KEY** puede aplicarse a una o más columnas.

5.2. EJEMPLOS DDL EN POSTGRESQL



Creación de Tablas y Restricciones Básicas

b) Único (**UNIQUE**)

La restricción **UNIQUE** asegura que todos los valores de una columna o combinación de columnas sean únicos en la tabla. Esto es útil para columnas como correos electrónicos o números de identificación.

```
correo VARCHAR(100) UNIQUE
```

c) No Nulo (**NOT NULL**)

La restricción **NOT NULL** impide que una columna acepte valores nulos, es decir, exige que la columna siempre tenga un valor, que sea obligatoria.

```
nombre VARCHAR(50) NOT NULL
```

5.2. EJEMPLOS DDL EN POSTGRESQL



Creación de Tablas y Restricciones Básicas

d) Verificación (**CHECK**)

La restricción **CHECK** permite aplicar condiciones personalizadas a los valores de una columna. Por ejemplo, asegurarse de que una columna de **salario** no tenga valores negativos.

```
salario NUMERIC(10, 2) CHECK (salario > 0)
```

5.2. EJEMPLOS DDL EN POSTGRESQL



Creación de Tablas y Restricciones Básicas

e) Clave Foránea (**FOREIGN KEY**)

Una **clave foránea** establece una relación entre dos tablas. **Su propósito es referenciar una columna en otra tabla, asegurando la integridad referencial entre ambas.** Al usar una clave foránea, podemos definir el comportamiento ante la eliminación o actualización de datos en la tabla referenciada.

- **Opciones para la cláusula **ON DELETE** y **ON UPDATE**:**

- **ON DELETE CASCADE:** Si un registro de la tabla referenciada se elimina, se eliminan todos los registros relacionados en la tabla actual.
- **ON DELETE SET NULL:** Si se elimina el registro en la tabla referenciada, el valor en la clave foránea de la tabla actual se convierte en **NULL**.
- **ON DELETE SET DEFAULT:** Si se elimina el registro en la tabla referenciada, el valor en la clave foránea se cambia al valor predeterminado de la columna.
- **ON DELETE RESTRICT:** Impide la eliminación de un registro en la tabla referenciada si hay registros relacionados en la tabla actual (por defecto).
- **ON DELETE NO ACTION:** Similar a **RESTRICT**, pero el comportamiento exacto puede depender de cómo el sistema de base de datos maneja internamente la transacción

5.2. EJEMPLOS DDL EN POSTGRESQL



Tabla Departamentos

Define una lista de departamentos con una clave primaria, un nombre único y una restricción de presupuesto para que solo permita valores positivos.

```
CREATE TABLE DEPARTAMENTOS(  
    id SERIAL PRIMARY KEY,  
    nombre VARCHAR(50) UNIQUE NOT NULL,  
    presupuesto NUMERIC(12, 2) CHECK (presupuesto > 0),  
    fecha_creacion DATE DEFAULT CURRENT_DATE  
);
```

- **id**: Clave primaria generada automáticamente.
- **nombre**: Campo único que no permite valores nulos.
- **presupuesto**: Restricción **CHECK** para aceptar solo valores mayores a 0.
- **fecha_creacion**: Fecha que por defecto se cargará con el día actual.

5.2. EJEMPLOS DDL EN POSTGRESQL



Tabla Empleados

Define una tabla para almacenar empleados, incluyendo una clave foránea que referencia a **departamentos**. Este diseño permite ver cómo los empleados están organizados por departamentos.

```
CREATE TABLE empleados (  
    id SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    salario NUMERIC(10, 2) CHECK (salario > 0),  
    fecha_contratacion DATE DEFAULT CURRENT_DATE,  
    departamento_id INT REFERENCES departamentos(id) ON DELETE SET NULL,  
    correo VARCHAR(100) UNIQUE CHECK (correo LIKE '%_@_%.__%'),  
    CONSTRAINT salario_min CHECK (salario >= 1000)  
);
```

- **Explicación de restricciones:**

- **PRIMARY KEY:** `id` actúa como identificador único.
- **UNIQUE:** `correo` debe ser único en cada registro.
- **CHECK:** `salario` tiene un mínimo de 1000 y un valor positivo.
- **FOREIGN KEY con ON DELETE SET NULL:** Si un departamento se elimina, el campo `departamento_id` se establece en `NULL` en la tabla `empleados`.

5.2. EJEMPLOS DDL EN POSTGRESQL



Tabla Proyectos

Cada proyecto se asocia con un departamento, por lo que utiliza una clave foránea que apunta a **departamentos**. Esto permite aplicar la restricción de cascada de eliminación en proyectos cuando el departamento relacionado se elimina.

```
CREATE TABLE proyectos (  
    id SERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    fecha_inicio DATE NOT NULL,  
    fecha_fin DATE CHECK (fecha_fin > fecha_inicio),  
    departamento_id INT NOT NULL REFERENCES departamentos(id) ON DELETE CASCADE,  
    CONSTRAINT proyecto_unico UNIQUE (nombre, departamento_id)  
);
```

- **Explicación de restricciones:**

- **CHECK:** La fecha de fin debe ser posterior a la fecha de inicio.
- **UNIQUE:** La combinación **nombre** y **departamento_id** debe ser única.
- **FOREIGN KEY** con **ON DELETE CASCADE:** Si un departamento se elimina, todos los proyectos asociados se eliminan automáticamente.

5.2. EJEMPLOS DDL EN POSTGRESQL



Tabla Asignaciones

Esta tabla crea una relación **muchos a muchos** entre **empleados** y **proyectos**, usando claves foráneas y restricciones para garantizar que cada asignación de empleado-proyecto es única.

```
CREATE TABLE asignaciones (  
    empleado_id INT REFERENCES empleados(id) ON DELETE CASCADE,  
    proyecto_id INT REFERENCES proyectos(id) ON DELETE CASCADE,  
    horas_semana INT CHECK (horas_semana > 0 AND horas_semana <= 40),  
    fecha_asignacion DATE DEFAULT CURRENT_DATE,  
    PRIMARY KEY (empleado_id, proyecto_id)  
);
```

- **Explicación de restricciones:**

- **PRIMARY KEY:** Combina **empleado_id** y **proyecto_id** para asegurar que cada par sea único.
- **CHECK:** Limita **horas_semana** a valores entre 1 y 40.
- **FOREIGN KEY con ON DELETE CASCADE:** Si se elimina un empleado o proyecto, todas las asignaciones relacionadas también se eliminan automáticamente.

5.2. EJEMPLOS DDL EN POSTGRESQL



Resumen de las Restricciones Aplicadas

Restricción	Uso	Ejemplo
PRIMARY KEY	Identificador único	<code>id SERIAL PRIMARY KEY</code>
UNIQUE	Valor único en la columna	<code>correo VARCHAR(100) UNIQUE</code>
NOT NULL	Impide valores nulos	<code>nombre VARCHAR(50) NOT NULL</code>
CHECK	Condición específica	<code>CHECK (salario > 0)</code>
FOREIGN KEY	Relación con otra tabla	<code>departamento_id INT REFERENCES departamentos(id)</code>
ON DELETE	Acción al eliminar en tabla padre	<code>ON DELETE CASCADE / SET NULL / RESTRICT</code>

5.3. Comando para ver la estructura de una tabla

Una vez que se ha creado una tabla en PostgreSQL, puedes utilizar el comando `\d` en la herramienta de línea de comandos `psql` para ver la estructura de la tabla. Este comando muestra información sobre las columnas, tipos de datos, restricciones y claves definidas en la tabla.

```
\d nombre_tabla
```

Ejemplo

Si tienes una tabla llamada `empleados`, ejecuta:

```
\d empleados
```

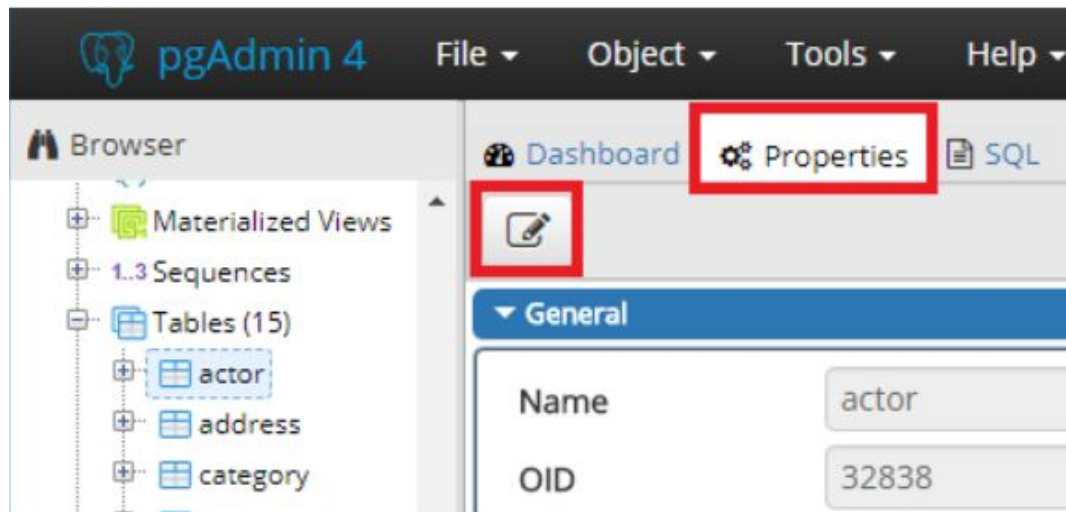
Este comando mostrará:

- Los nombres de las columnas.
- Los tipos de datos de cada columna.
- Las restricciones aplicadas, como claves primarias, foráneas y restricciones `CHECK`.

5.3. Comando para ver la estructura de una tabla

Otras Alternativas

Si estás utilizando **pgAdmin**, puedes ver la estructura de la tabla navegando hasta ella en el panel de navegación, y seleccionando la opción **Columnas** o **Properties** (Propiedades) para ver los detalles de su estructura.



5.5. EJEMPLOS DDL EN POSTGRESQL (MODIFICACIONES)



Modificación de Tablas (ALTER TABLE)

Con **ALTER TABLE**, puedes añadir, modificar o eliminar columnas de una tabla.

- **Añadir columna:**

```
ALTER TABLE empleados ADD COLUMN direccion VARCHAR(100);
```

- **Modificar columna:**

```
ALTER TABLE empleados ALTER COLUMN salario TYPE INTEGER;
```

- **Eliminar columna:**

```
ALTER TABLE empleados DROP COLUMN direccion;
```

5.5. EJEMPLOS DDL EN POSTGRESQL (BORRADO)



- **Eliminación de Tablas (DROP TABLE)**

Elimina por completo la tabla y su contenido.

```
DROP TABLE IF EXISTS empleados;
```

- **Eliminación de todos los datos de la Tabla (TRUNCATE TABLE)**

Elimina el contenido de la tabla, pero mantiene su estructura. Es decir, deja la tabla vacía.

```
TRUNCATE TABLE empleados;
```

5.5. EJEMPLOS DDL EN POSTGRESQL (DROP)

El comando **DROP** elimina una tabla (o cualquier otro objeto de base de datos, como vistas, índices o esquemas) junto con todos sus datos y estructura de forma permanente. Una vez ejecutado, no se puede recuperar la tabla ni sus datos, a menos que exista una copia de seguridad.

Sintaxis:

```
DROP TABLE nombre_tabla;
```

Ejemplo:

`DROP TABLE empleados;` → Este comando elimina la tabla `empleados` de la base de datos junto con todos los datos almacenados en ella.

- **Opciones adicionales:**

- **IF EXISTS:** Evita un error si la tabla no existe. `DROP TABLE IF EXISTS empleados;`
- **CASCADE:** Elimina la tabla y todas las dependencias, como claves foráneas en otras tablas que la referencian. `DROP TABLE empleados CASCADE;`
- **RESTRICT:** Previene la eliminación si existen objetos dependientes (predeterminado).

5.5. EJEMPLOS DDL EN POSTGRESQL (TRUNCATE)

El comando **TRUNCATE** elimina todos los datos de una tabla sin eliminar su estructura. Es una operación rápida y eficiente, ideal para borrar grandes cantidades de datos, ya que no genera un registro de cada eliminación individual (lo que sí hace el comando **DELETE**). Sin embargo, **no se puede revertir** (no genera un registro por cada fila eliminada).

Sintaxis: `TRUNCATE TABLE nombre_tabla;`

Ejemplo: `TRUNCATE TABLE empleados;` → Este comando elimina todas las filas de la tabla `empleados`, pero conserva la estructura de la tabla.

- **Opciones adicionales:**
 - **CASCADE:** También vacía las tablas que tienen claves foráneas referenciadas en la tabla `empleados`.
`TRUNCATE TABLE empleados CASCADE;`
 - **RESTART IDENTITY:** Restaura los valores de columnas **SERIAL** o **IDENTITY** a su valor inicial.
`TRUNCATE TABLE empleados RESTART IDENTITY;`
 - **CONTINUE IDENTITY:** Mantiene el último valor de las columnas **SERIAL** o **IDENTITY** (predeterminado).

5.5. EJEMPLOS DDL EN POSTGRESQL (VISTAS)



Las vistas permiten crear "tablas virtuales" basadas en consultas.

- **Crear una vista:**

```
CREATE VIEW vista_empleados_activos AS  
SELECT nombre, puesto, salario  
FROM empleados  
WHERE es_activo = TRUE;
```

- **Eliminar una vista:**

```
DROP VIEW IF EXISTS vista_empleados_activos;
```

5.5. EJEMPLOS DDL EN POSTGRESQL (ÍNDICES)



Los índices optimizan las consultas en columnas específicas.

- **Crear un índice:**

```
CREATE INDEX idx_nombre_empleado ON empleados(nombre);
```

- **Índice único:**

```
CREATE UNIQUE INDEX idx_correo_empleado ON empleados(correo);
```

- **Eliminar un índice:**

```
DROP INDEX IF EXISTS idx_nombre_empleado;
```

6. DICCIONARIO DE DATOS EN POSTGRESQL

En PostgreSQL, toda la información sobre las estructuras de la base de datos, como tablas, columnas, restricciones y claves, se almacena en el **diccionario de datos**. Este diccionario contiene varias tablas del esquema `pg_catalog`, donde PostgreSQL guarda los metadatos. Aquí se detallan las tablas más relevantes

Tablas Principales del Diccionario de Datos de PostgreSQL

Tabla	Descripción
<code>pg_tables</code>	Contiene información sobre las tablas de usuario y del sistema.
<code>pg_class</code>	Almacena información sobre todas las tablas, vistas, índices y secuencias.
<code>pg_attribute</code>	Guarda información sobre las columnas de las tablas, como nombre y tipo de dato.
<code>pg_constraint</code>	Almacena las restricciones de las tablas, como claves primarias y foráneas.
<code>pg_index</code>	Contiene información sobre los índices definidos en las tablas.
<code>pg_enum</code>	Almacena los valores definidos para tipos de datos enumerados (<code>ENUM</code>).
<code>pg_type</code>	Guarda información sobre los tipos de datos definidos en PostgreSQL.
<code>pg_roles</code>	Lista todos los roles y permisos de usuario en la base de datos.
<code>pg_namespace</code>	Almacena los esquemas definidos en la base de datos.



6. DICCIONARIO DE DATOS EN POSTGRESQL

1. Listar Tablas Creadas en un Esquema

Para ver las tablas definidas por el usuario (excluyendo las de sistema) en el esquema actual:

```
SELECT tablename  
FROM pg_tables  
WHERE schemaname = 'public';
```



6. DICCIONARIO DE DATOS EN POSTGRESQL

2. Ver Columnas de una Tabla (pg_attribute)

La tabla `pg_attribute` en combinación con `pg_class` te permite ver las columnas y sus tipos de datos en una tabla específica.

```
SELECT a.attname AS columna, t.typname AS tipo_dato, a.attnotnull AS not_null
FROM pg_attribute a
JOIN pg_class c ON a.attrelid = c.oid
JOIN pg_type t ON a.atttypid = t.oid
WHERE c.relname = 'empleados' AND a.attnum > 0;
```

- **Resultado esperado:** Te mostrará cada columna de `empleados`, su tipo de dato y si es `NOT NULL`.



6. DICCIONARIO DE DATOS EN POSTGRESQL

3. Ver Restricciones de una Tabla (pg_constraint)

Para consultar restricciones como claves primarias, únicas, **CHECK**, y claves foráneas, usa `pg_constraint` junto con `pg_class`:

```
SELECT conname AS restriccion, contype AS tipo, conrelid::regclass AS tabla  
FROM pg_constraint  
WHERE conrelid::regclass = 'empleados'::regclass;
```

- **Tipo de Restricción (contype):**

- **p**: Clave primaria.
- **u**: Clave única.
- **c**: Restricción **CHECK**.
- **f**: Clave foránea.



6. DICCIONARIO DE DATOS EN POSTGRESQL

4. Ver Índices en una Tabla (pg_index)

Para ver los índices creados en una tabla específica, usa `pg_index` junto con `pg_class`:

```
SELECT indexrelid::regclass AS nombre_indice, indrelid::regclass AS tabla
FROM pg_index
WHERE indrelid::regclass = 'empleados'::regclass;
```

- **Resultado esperado:** Lista los índices en la tabla `empleados`.



6. BIBLIOGRAFÍA

- **Date CJ: Introducción a los sist. de B.D. Ed. Addison-Wesley**
- **De Miguel A. y Piattini M. Fundamentos de los Modleos de BBDD. Ed Ra-Ma**
- **Kolth y Silberhatz. Fundamentos de B.D. Ed. Mc Graw Hill.**
- **Lopez Montalban, I. Admono de SGBD. Ed. Garceta**
- <https://jorgesanchez.net/manuales/sql/ddl-tablas-sql2016.html>
- <https://www.youtube.com/watch?v=gEJcMrk3E-Q&list=PLgqdACsQ8US2DCzQVrdZDZCTtTFHMY0as&index=1>
-